QUESTION3:

1)A quadrature encoder has two sensors. They use the slots in the wheels and the timings of pulse receivals to find out the speed of the motor's rotation and the no.of rotations too. The main advantage of quadrature encoder lies in its ability to distinguish between the direction in which the motor spins. The two sensors are placed such that they are physically out of phase(90deg shift in the square wave form graph they give) thus on the basis of which sensor gets a HIGH signal first we can estimate the direction (forward/backward) of the wheel.
By the given data we can say that for one motor shaft revolution 500*4 pulses are emitted. Also 30 revolutions of the motor shaft equals one revolution of the wheel. So for one wheel revolution we receive 60,000 pulses. Circumference of the wheel=π*15=47.124cm, thus the wheel would move 47.124 cm for 60,000 counts-> 0.0007854cm per count.

2)
```
from math import sin,cos,degrees
import matplotlib.pyplot as plt
coordinates=[(0.0,0.0,degrees(0.0))]
x=0
y=0
theta=0
for i in range(4):
    left=0.0001*int(input('LEFT:'))
    right=0.0001*int(input('RIGHT:'))
    arclength=(left+right)/2
    delta_theta=(right-left)/0.5 #wheelbase
    theta_mid=theta+delta_theta/2
    x+=arclength*cos(theta_mid)
    y+=arclength*sin(theta_mid)
    theta+=delta_theta
    coordinates.append((x,y,degrees(theta)))
x_coord=[coordinate[0] for coordinate in coordinates]
y_coord=[coordinate[1] for coordinate in coordinates]
plt.figure(figsize=(8,8))#draws the plane
plt.plot(x_coord,y_coord,marker='o',linestyle='-',color='b')#the real
plotting
plt.axhline(0,color='grey')
plt.axvline(0,color='grey')
plt.grid(True)#makes bg grid visible
plt.show()
```

This is the code I wrote, but my defender blocks it. Couldn't rectify it by the deadline.

3)Since the right wheel is 1% larger it would cover 1% more distance than the left wheel that is 10.1m. The angle swept by the radial vector would be 0.1/0.5 radians that is 0.2. So when the encoder thinks the bot has gone in a straight line, it has actually deviated off course by 0.2 radians.
Three other Errors would:
Drifting of wheels on the surface due to which distance calculation wouldn't accurately map to revolutions per count*count*circumference.
Unevenly distributed weight such as the battery pack on the left side alone could cause the motor on the left side to seek more power than that used by the right side, if equal power was given then the bot could deviate left.
Not switching both the motors on simultaneously can also cause such deviations.